

Article

Industrial Robot Control System with a Predictive Maintenance Module Using IIoT Technology

Andrzej Wojtulewicz ^{*,†}  and Patryk Chaber [†] 

Institute of Control and Computation Engineering, Faculty of Electronics and Information Technology, Warsaw University of Technology, ul. Nowowiejska 15/19, 00-665 Warsaw, Poland; patryk.chaber@pw.edu.pl

* Correspondence: andrzej.wojtulewicz@pw.edu.pl

† These authors contributed equally to this work.

Abstract: The article describes solutions in the field of diagnostics of a control system based on a CNC and the cooperation with an industrial robot. The industrial robot is controlled directly from the CNC. Data exchange between the CNC and the robot controller allows for collecting the most important process data from the robot. Then, calculations are performed in the PLC using a number of functions to obtain consumption indicators of individual robot components. The data were visualized on the HMI screens of the CNC. Additionally, a dedicated interface was prepared to share these data using the MQTT protocol for IIoT solutions. The entire solution was implemented and then deployed in a real station. The presented solution is an extension of the possibilities of operating an industrial robot by CNC towards diagnostics and early failure prevention.

Keywords: industrial robot; diagnostics; prediction; IIoT; CNC; HP HMI; MQTT; communication protocols



Academic Editor: Andrey V. Savkin

Received: 13 January 2025

Revised: 29 January 2025

Accepted: 12 February 2025

Published: 13 February 2025

Citation: Wojtulewicz, A.; Chaber, P. Industrial Robot Control System with a Predictive Maintenance Module Using IIoT Technology. *Sensors* **2025**, *25*, 1154. <https://doi.org/10.3390/s25041154>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The work presented in the article concerns the diagnostics of consumption of individual parts of an industrial robot controlled by a CNC (Computerized Numerical Control). In current industrial revolution era, there is a visible trend to invest in advanced systems for continuous diagnostics using this type of equipment. Special measuring systems are used in combination with analytical software. Advanced algorithms are used to process measurement data to predict possible failures. This allows for planning maintenance and interventions in advance, instead of reacting to problems that already occur. Thanks to the early detection of potential problems, the maintenance department can plan maintenance and repairs in an organized and efficient manner, minimizing downtime. This planning can be integrated with other management systems, allowing for resource management. Based on the analysis, the system generates alarms for the maintenance department. Alarms can inform us about the need to carry out inspection, maintenance, or repair. In the long term, the collected data and conclusions from the predictive diagnostics system can be used to optimize production processes and improve machine design. These systems also enable a better understanding of the causes of failures and contribute to the continuous improvement of operational processes. The article presents implementations of solutions based on the PLC (Programmable Logic Controller) and the analysis of data from the robot. In addition, the prepared interface allows for data collection using IIoT (Industrial Internet of Thing) technology for superior IT (Information Technology) systems.

The following part of the chapter presents examples from the literature concerning the topic of the article. The second chapter presents the general structure of the system

as well as the most important elements used in the implementation of the work. The IIoT technology used to provide diagnostic data is described. Then, the implementations of the most important diagnostic functions implemented in the PLC calculated on the basis of process data from the robot are described. The third chapter presents the implemented visualization screens with sample data collected from the actual implementation of the robot in the station. The last chapter summarizes the most important results obtained during the work.

1.1. Related Work

The aim of Ref. [1] is to present a practical implementation of a network architecture capable of collecting data from industrial robots, in order to later implement “One-Class Novelty Detection” models—identification of data that significantly differ from patterns observed during machine learning. The architecture uses only well-known automation components, such as a PLC. The implementation was performed on a real assembly line in the automotive industry, where it was possible to demonstrate the effectiveness of detecting anomalies in the work of robots.

Ref. [2] describes the process of several possible solutions for predictive diagnostics of robots from data collection, through the creation of an appropriate data set, to the use of machine learning algorithms. The algorithms were evaluated using appropriate performance metrics in terms of accuracy and precision, which were proven to be effective in predicting robot damage. Additionally, the impact of the Principal Component Analysis (PCA) technique on the obtained results, used to reduce the dimensionality of the data, was checked.

Ref. [3] presents a methodology for creating a data acquisition strategy for predictive diagnostics of industrial robots. The paper describes the components of six-axis robots that are susceptible to failure in the automotive industry, such as power supplies, servos, or gears, and it also presents data sources that are a source for detecting faults of robot components. The most commonly used parameters for fault detection are motor currents and vibration analysis.

Ref. [4] presents the application of an LSTM recurrent neural network for predictive diagnostics of industrial robots. The model predicts the day of failure based on alarm analysis, achieving high prediction accuracy. The significant help this solution provides to the maintenance department in effective maintenance planning is described.

Ref. [5] developed a predictive diagnostic model for industrial robots based on an artificial neural network, using MTTF values and system failure history. Data were acquired using an ERP system, and the results showed the effectiveness of predicting failures by the MLP structure, minimizing costs related to unplanned downtime. Reliability analysis and spare part forecasts based on the Poisson distribution allowed for the creation of an optimal spare part list, ensuring production efficiency is at 85%.

In Ref. [6], an approach based on the analysis of robot currents is proposed to detect accuracy errors of individual robot joints for predictive diagnostics. The methodology is based on the analysis of the time series of currents to predict deviations in robot accuracy. The results confirmed a strong correlation between the selected three-phase current properties and robot accuracy. The developed predictive model showed high efficiency and good fit.

In Ref. [7], a predictive maintenance method for semiconductor wafer transport robots was developed using the K-means clustering algorithm to analyze data from acceleration sensors attached to the robot arm. The data were subjected to normalization and denoising processes, and the data analysis enabled the prediction of the appropriate maintenance moment. Simulations showed that the method allows for real-time prediction, minimizing the risk of sudden failures.

The aim of Ref. [8] was to propose a robotic cell reliability optimization method based on the digital twin (DT). A machine learning model was used to detect and classify faults of critical components and predict their remaining service life (RUL). Reliability analysis using RBD and FTA methods showed that parallel connections of critical components minimize the risk of failure of the entire system.

Ref. [9] concerns the implementation of predictive diagnostics for industrial robots, only using data from existing sensors, without the need to install new ones. Torque characteristics were used to predict robot anomalies. Different algorithms were tested and the results showed that the hybrid approach (Exponentron + GPR) is the best. It was also shown that low data quality significantly worsens the results, so it is necessary to have high-quality data and a good sampling rate.

Ref. [10] concerns predictive fault detection in robots using motor current signal analysis (MCSA), aimed at monitoring and assessing the condition of the robot drive system. The applied method allows for fault detection in the drive system. The novelty is in mastering the classical limitations of signal analysis in the frequency domain, extending the range beyond the steady state. Signals outside the steady states showed rich information content.

Ref. [11] presents reliability engineering methods that can be used to predict the failure probability of mobile robots. A new modification of the mean time to failure concept is proposed, taking into account the influence of operating conditions on the failure rate. Furthermore, these techniques are applied to the design of mobile robot tasks, enabling the prediction of the probability of task completion.

1.2. Article Contribution

The article presents the implementation of a diagnostic system for an industrial robot controlled directly from the CNC [12]. The robot performs loading tasks [13] of a workpiece for automatic machining [14]. In order to ensure the highest production reliability [15], diagnostic solutions [16] were presented, allowing for the earlier response of maintenance services with regard to servicing and ensuring the continuity of the entire system [17]. Based on the data collected from the industrial robot [18], the consumption rates of individual parts are calculated, thanks to which it is possible to prevent system failures, thus reducing the costs related to the unplanned downtime of the station. Dedicated HMI (Human Machine Interface) screens have been designed on the CNC, which allow for insight into current data from the industrial robot. Maintenance services can use these data to plan the replacement of a specific assembly or perform service activities related to lubrication in advance. Implementations in the PLC and the visualization of HMI panels have been presented. To increase functionality and adapt to the new standards of Industry 4.0 [19], the system has been supplemented with the ability to download data via MQTT (Message Queue Telemetry Transport) [20,21] for the needs of IIoT solutions [22].

The most important features of the presented solution are as follows:

- Extending the scope of access to the diagnostic data of the CNC system, in particular, through the MQTT broker [23] for IIoT technology [24].
- Expanding the robot control mechanism with diagnostic functions and predicting the servicing of robot parts [25].
- The implementation made in the PLC, which eliminates the need for additional devices. Possible implementation on analogous control systems [26].
- The visualization of all diagnostic information directly on the CNC. Additionally, making these data available through IIoT [27].
- The possibility of collecting other process data regarding machining on the CNC machine.

- The functions for prediction are open and can be freely modified and further developed for robots from other manufacturers.
- The system is open to the possibility of communication with robots from other manufacturers, e.g., FANUC.
- The developed solution can be extended for use with any devices (sensors or controllers) supporting the SLMP. It is also possible to easily modify the implementation with an additional MODBUS TCP protocol using Ethernet [28].

2. Materials and Methods

The article presents the most interesting fragments of programs implemented in the PLC calculating consumption. The implementation was made in the ST (Structured Text) language. From the point of view of PLC programming, this is a dedicated solution for performing calculations.

The structure of the system is shown in Figure 1. The main CNC system M80V Mitsubishi Electric performs the task of controlling the lathe in the first channel. The second channel is dedicated to controlling the Mitsubishi Electric RV2F industrial robot. The CR800D robot controller is connected to the CNC system via Ethernet, thanks to which it is possible to send all the necessary process data (CC-Link IE Field Basic) from the robot as well as its control (Direct Robot Control). For the convenient setting of the robot's operating parameters and for performing its diagnostics, dedicated screens displayed directly on the CNC have been prepared. NC Designer 2 software version 2.1.9 was used to implement the visualization. Access to the CNC screens is possible using VNC (Virtual Network Computing). To obtain IIoT functionality, a dedicated PC was used, which has an MQTT broker providing data from the field of industrial robot diagnostics. The broker is supplied with data using the SLMP, which retrieves data directly from the CNC's PLC. As an alternative to a PC, it is possible to use a microcontroller with analogous functions, which has extended the scope of the access to diagnostic data of the CNC. In addition to the implementation of diagnostic functions, the robot's learning and trajectory playback functions have also been implemented. The operator's safety is ensured by a dedicated safety system for which software for redundant PLCs has been developed. It is also possible to use an optional vision system for the inspection of the manufactured part.

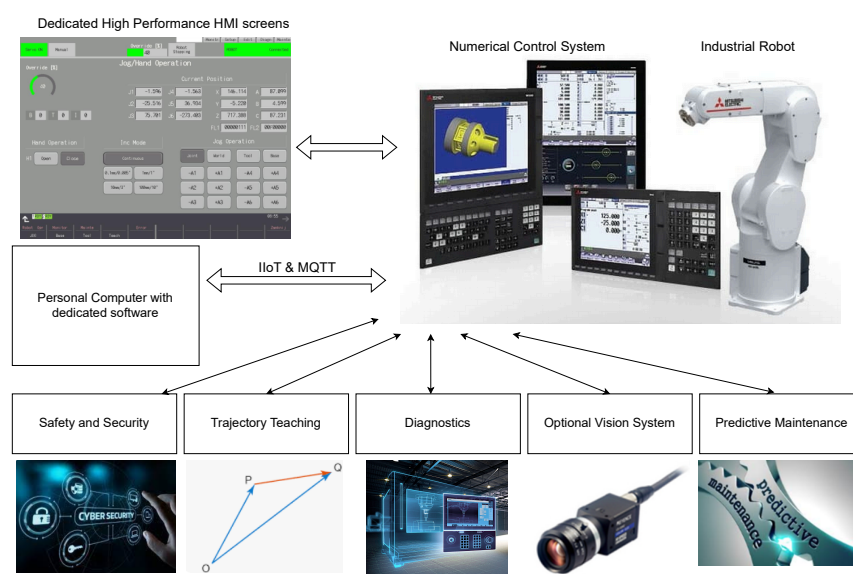


Figure 1. Control system structure.

2.1. IIoT Interface

MQTT is a communication protocol designed for resource-constrained devices in Industrial Internet of Things (IIoT) solutions. It was designed for use with small data sizes and high-latency flows. It has low requirements, and it is reliable and efficient. Thanks to this, it has become a basic element in IIoT solutions. It can be used in IIoT devices that run on batteries, sending data to collecting and processing servers. This solution is ideal for devices that require low power consumption and minimal data transfer makes it suitable for IIoT applications. The principle of operation of the MQTT protocol is based on the principle of publishing and subscribing messages. The device publishing data sends them to a specific topic on the MQTT broker, which acts as a center for managing the flow of messages. Other devices can subscribe to specific topics and receive any data sent to them. An important aspect of MQTT is its implementation efficiency—the small size of the communication frame base and its high compression allow for fast and reliable communication between devices.

The dataflow of data used in the presented solution is shown in Figure 2. Sending important information about current process values from the robot controller to the PLC was done using CC-Link IEF Basic communication. Furthermore, some data were sent using the DRC (Direct Robot Control) protocol used, in general, to control the connected robot. All of the data were displayed on dedicated screens, and some of them were taken to calculating diagnostic coefficients.

The SLMP (Seamless Message Protocol) is a communication protocol used by Mitsubishi Electric. It was developed to facilitate communication between industrial automation devices. The CC-Link IEF Basic protocol used to exchange data between the PLC and the robot controller is based on the SLMP. It is characterized by fast and reliable communication. It works in client-server mode, the client (PC or microcontroller) sends a request to the server, and the server (PLC) processes the request and sends a response. The main functionality of the SLMP connection used in this application was reading data, i.e., reading specific areas of the PLC memory. For the CNC, it is necessary to configure parameter #1489 – SLMP_ON to 1, activating the SLMP server on the PLC.

Robot data that were sent to the PLC or data that were calculated in the PLC were sent to the PC or microcontroller, which acted as a protocol converter. The communication was done with the PLC via the SLMP, reading the necessary data, and then preparing them using the MQTT communication protocol. In this way, the data were prepared for master devices using IIoT technology. The solution does not include security mechanisms against cyberattacks. The communication protocols and Ethernet connections were used in their standard versions and access protection was left to external network devices and the network administrator.

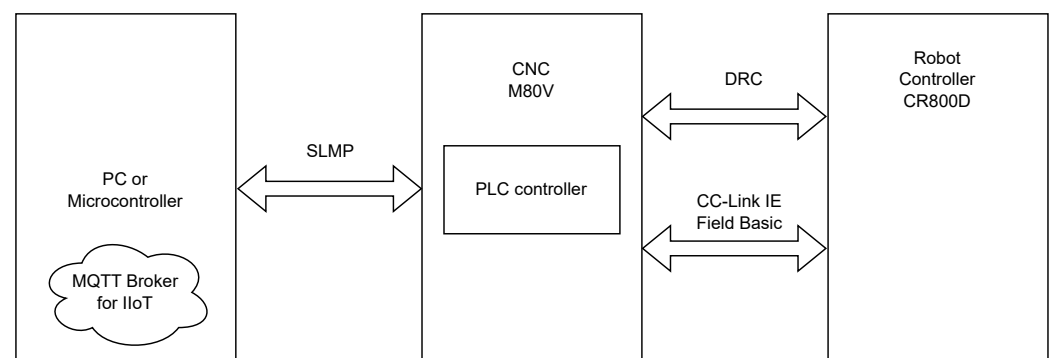


Figure 2. Dataflow for IIoT.

2.2. Grease Consumption

Grease consumption is calculated based on the speed of the motor in a given joint. The time interval specified in the settings can become longer or shorter. Consumption is not calculated if the motors are not running. The function block presented in Listing 1 is responsible for calculating the consumption of grease. The following calculations are carried out according to the block diagram shown in Figure 3.

If the current speed of the motor is different from zero, the variable RealTimeMs is incremented every 100 ms, which counts the time the lubricant is running without any penalty. Then, the percentage of the current motor speed relative to the maximum speed is calculated. This value in percentage is passed as an argument to the quadratic functions. The function for an argument above 50% returns values greater than 100 ms, while for an argument below 50%, it returns values less than 100 ms. This means that operation at speeds above 50% is penalized, while at speeds below 50% is rewarded, extending the grease replacement interval. Grease consumption is calculated based on the time generated using quadratic functions. By comparing this time with the real grease operation time, it is possible to estimate the grease replacement interval based on the consumption to date and calculate the remaining hours to be serviced with such robot operations.

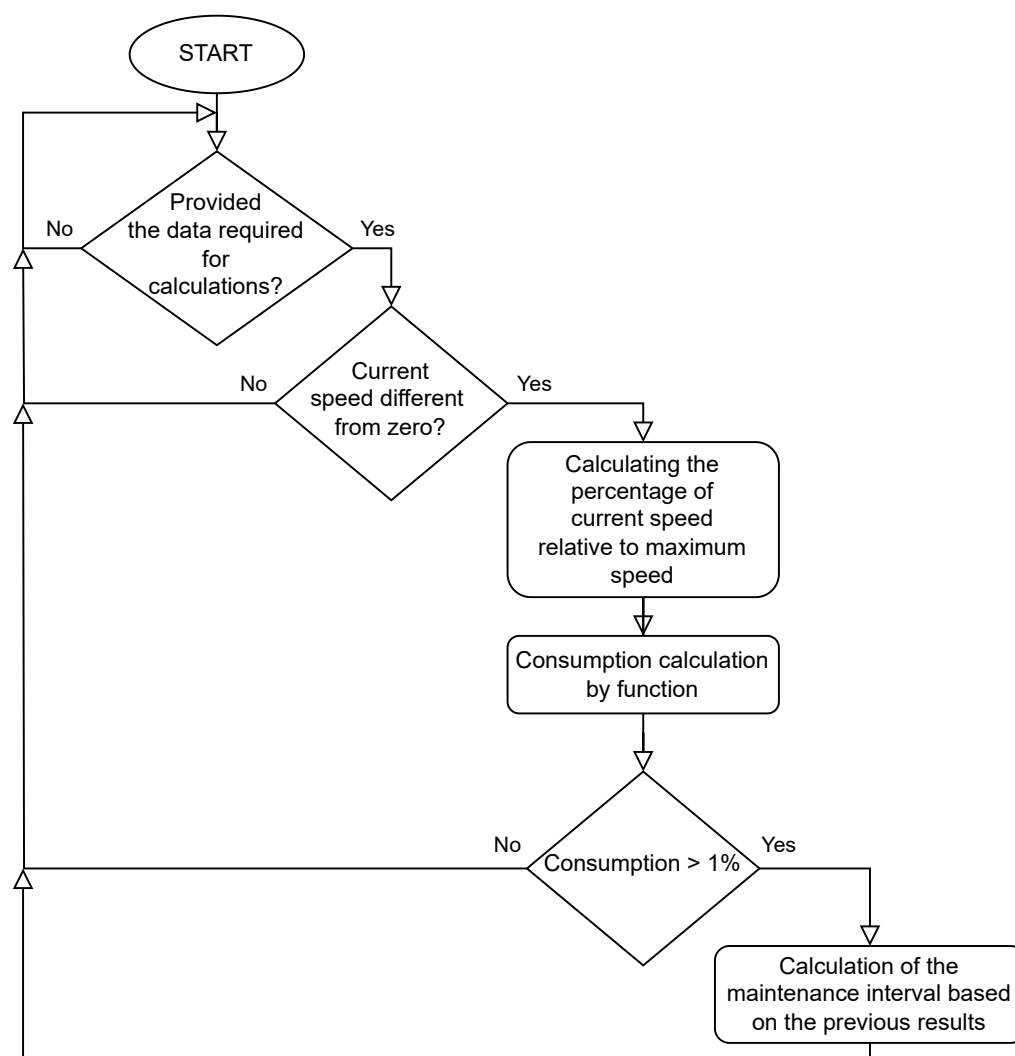


Figure 3. Grease consumption function flow diagram.

Listing 1. Robot consumption grease function block.

```

1 IntervalSec := INT_TO_DINT(IntervalHours) * 3600;
2
3 IF IntervalSec > 0 AND MaxJointSpeed > 0 THEN
4   (*Checking if the current speed is different from zero*)
5   IF RobotSts.bRobotConnectionStatus AND RobotSts.bRobotMotionStatus AND
      MotorSpeed <> 0 AND LDP(TRUE, SM410) THEN
6     (*Real time counting*)
7     DINC(TRUE, RealTimeMs);
8
9     (*Converting speed to percentage of maximum*)
10    ActualSpeedPercent := LREAL_TO_REAL(DINT_TO_LREAL(ABS(ActualSpeed)) / (
        INT_TO_LREAL(MaxJointSpeed) * 1000.0) * 100.0);
11
12    (*Calculation of the function value*)
13    IF ActualSpeedPercent >= 50.0 THEN
14      FunctionValueMs := (0.01 * ((ActualSpeedPercent - 50.0) * (ActualSpeedPercent - 50.0))) + 100.0;
15    ELSIF ActualSpeedPercent < 50.0 THEN
16      FunctionValueMs := (-0.01 * ((ActualSpeedPercent - 50.0) * (ActualSpeedPercent - 50.0))) + 100.0;
17    END_IF;
18
19    (*Adding time result*)
20    TimeMs := TimeMs + FunctionValueMs;
21
22    (*Checking for full seconds*)
23    IF TimeMs > 1000.0 THEN
24      DINC(TRUE, TimeSec);
25      TimeMs := TimeMs - 1000.0;
26    END_IF;
27  END_IF;
28
29  (*Calculation of consumption in percentage*)
30  ConsumptionPercent := LREAL_TO_REAL((DINT_TO_LREAL(TimeSec) / DINT_TO_LREAL(IntervalSec)) * 100.0);
31
32  (*Calculation of hours to service*)
33  IF ConsumptionPercent > 1.0 THEN
34    (*Interval according to previous consumption*)
35    EstIntervalTimeSec := LREAL_TO_DINT(DINT_TO_LREAL(IntervalSec) * (((
        DINT_TO_LREAL(RealTimeMs) / 10.0) / DINT_TO_LREAL(TimeSec))));
36
37    (*Remaining hours*)
38    EstRemainHours := LREAL_TO_INT((DINT_TO_LREAL(EstIntervalTimeSec) - (
        DINT_TO_LREAL(RealTimeMs) / 10.0)) / 3600.0);
39  ELSE
40    EstRemainHours := IntervalHours;
41  END_IF;
42 END_IF;
43
44 (*Consumption Maintenance Parts*)
45 IF EstRemainHours <= MaintPartsTime THEN
46   MaintPartsTime := EstRemainHours;
47   MaintPartsPercent := ConsumptionPercent;
48 END_IF;

```

2.3. Timing Belt Consumption

Timing Belt consumption is calculated based on the value of the currents and the rate of change. The time interval specified in the settings may not become longer, but it may become shorter. Using the robot at high speeds and with sudden changes in direction will result in a shorter interval. Consumption is calculated from the moment the Servo On is turned on, if the currents are different from zero—consumption can also be calculated while the robot is stationary. The function block presented in Listing 2 is responsible for calculating the consumption of the timing belt. The following calculations are carried out according to the block diagram shown in Figure 4.

If the current is different from zero, the variable RealTimeMs is incremented every 100 ms, which counts the belt running time without any penalty. Then, the percentage of the current value relative to the maximum current value is calculated. This value in percentage is passed as an argument to a quadratic function. The function for an argument above 20% returns values greater than 100 ms, while for an argument below 20%, it returns a value of 100 ms. This means that consumption increases when 20% of the current is exceeded. In addition, the value of the change in current as a percentage of the maximum current is calculated. If the value exceeds 5%, then, additional ms are added using a linear function. The values from both functions are summed. Belt consumption is calculated based on the time generated by the functions. By comparing this time with the real time of belt operation, it is possible to estimate the replacement interval on the basis of the consumption to date and calculate the remaining hours to be serviced with such robot operations.

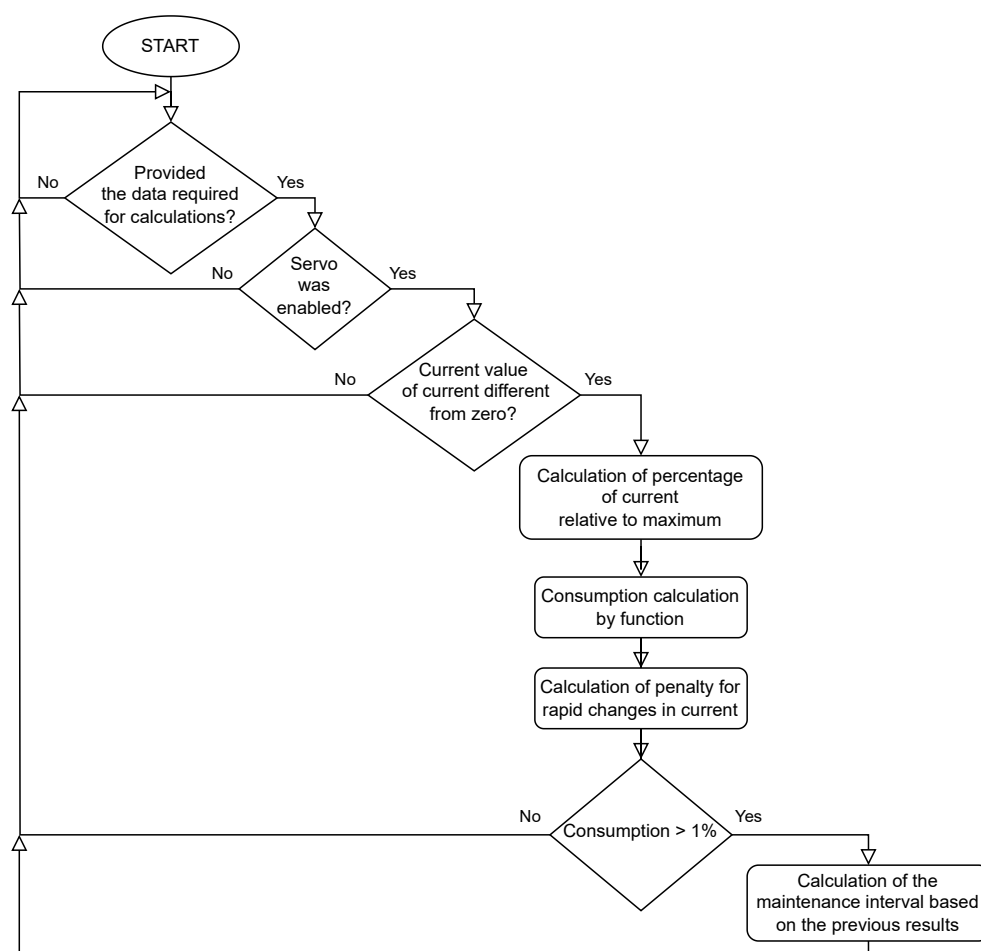


Figure 4. Timing belt consumption function flow diagram.

Listing 2. Timing belt consumption function block.

```

1 IntervalSec := INT_TO_DINT(IntervalHours) * 3600;
2
3 IF IntervalSec > 0 AND MaxTolerableCurrent > 0 THEN
4   (*Check if servo is ON*)
5   IF RobotSts.bRobotConnectionStatus
6   AND RobotSts.bRobotDriveStatus
7   AND ActualCurrent <> 0
8   AND LDP(TRUE, SM410) THEN
9     (*Real time counting*)
10    DINC(TRUE, RealTimeMs);
11
12    (*Converting current to percentage of maximum*)
13    ActualCurrentPercent := LREAL_TO_REAL((DINT_TO_LREAL(ABS(ActualCurrent))/
14      INT_TO_LREAL(MaxTolerableCurrent))*100.0);
15
16    (*Calculation of the function value*)
17    IF ActualCurrentPercent >= 20.0 THEN
18      FunctionValueMs := (0.0125*((ActualCurrentPercent-20.0)*(
19        ActualCurrentPercent-20.0)))+100.0;
20    ELSIF ActualCurrentPercent < 20.0 THEN
21      FunctionValueMs := 100.0;
22    END_IF;
23
24    (*Calculation of the percentage change of the maximum*)
25    ActualCurrentShiftPercent := LREAL_TO_REAL((DINT_TO_LREAL(ABS(
26      ActualCurrent - LastMeasureCurrent))/INT_TO_LREAL(MaxTolerableCurrent)
27      )*100.0);
28
29    (*Calculating the penalty for quick changes*)
30    IF ActualCurrentShiftPercent >= 5.0 THEN
31      FunctionFastShiftMs := (ActualCurrentShiftPercent-5.0);
32    ELSIF ActualCurrentShiftPercent < 5.0 THEN
33      FunctionFastShiftMs := 0.0;
34    END_IF;
35
36    (*Adding time result*)
37    TimeMs := TimeMs + FunctionValueMs + FunctionFastShiftMs;
38
39    (*Saving past value*)
40    LastMeasureCurrent := ActualCurrent;
41  END_IF;
42
43  (*Calculation of consumption in percentage*)
44  ConsumptionPercent := LREAL_TO_REAL((DINT_TO_LREAL(TimeSec)/DINT_TO_LREAL(
45    IntervalSec))*100.0);
46
47  (*Calculation of hours to service*)
48  IF ConsumptionPercent > 1.0 THEN
49    (*Interval according to previous consumption*)
50    EstIntervalTimeSec := LREAL_TO_DINT(DINT_TO_LREAL(IntervalSec) * (((
51      DINT_TO_LREAL(RealTimeMs)/10.0)/DINT_TO_LREAL(TimeSec))));
52
53    (*Remaining hours*)
54    EstRemainHours := LREAL_TO_INT((DINT_TO_LREAL(EstIntervalTimeSec) - (
55      DINT_TO_LREAL(RealTimeMs)/10.0)) / 3600.0);
56  ELSE
57    EstRemainHours := IntervalHours;
58  END_IF;
59 END_IF;

```

2.4. Gear Consumption

Gear consumption is calculated based on the current values, the rate of their change, and the motor speed. The time interval specified in the settings cannot be extended, but it may be shortened. Using the robot at high speeds and with sudden changes in direction will result in a shortened interval. Consumption is counted from the moment Servo On is turned on, if the currents are different from zero—consumption can also be counted when the robot is stationary.

If the current is different from zero, the RealTimeMs variable is incremented every 100ms, which counts the gear operation time without any penalties. Then, the percentage of the current current value relative to the maximum is calculated. This value of the percentage is passed as an argument of the quadratic function. For an argument above 20%, the function returns values greater than 100 ms, while for an argument below 20%, it returns the value 100 ms. This means that the consumption increases after exceeding 20% of the current. In addition, the value of the current change is calculated as a percentage of the maximum current. If this value exceeds 5%, then, additional ms are added using a linear function. Then, the percentage of the current engine speed relative to the maximum is calculated. This percentage value is passed as an argument of the quadratic function. For an argument above 30%, the function returns values greater than 0 ms, while for an argument below 30%, it returns the value 0 ms. The values from the three functions are added together. The consumption of the gear unit is calculated based on the time generated by the functions. Comparing this time with the real working time of the gear unit allows us to estimate the replacement interval, based on the current consumption, and calculate the remaining hours for inspection for such robot operations.

2.5. Power on Time, Servo on Time

The function presented in Listing 3 is used to calculate the working time of the robot and the working time of the drives of individual joints. It is possible to set the allowable number of hours of operation of a given element. After counting the set number of hours, an appropriate message will be appear on the alarm screen, suggesting the performance of the appropriate service actions.

Listing 3. Power on time.

```

1 IF RobotSts.bRobotConnectionStatus AND LDP(TRUE, SM410) THEN
2 INC(TRUE, RobotMaintenance.uPowOnTimeSec);
3 IF RobotMaintenance.uPowOnTimeSec = 600 THEN
4 RobotMaintenance.uPowOnTimeSec := 0;
5 INC(TRUE, RobotMaintenance.uPowOnTimeMin);
6 IF RobotMaintenance.uPowOnTimeMin = 60 THEN
7 RobotMaintenance.uPowOnTimeMin := 0;
8 INC(TRUE, RobotMaintenance.uPowOnTime);
9 END_IF;
10 END_IF;
11 END_IF;
12 IF RobotSts.bRobotConnectionStatus AND RobotSts.bRobotDriveStatus AND LDP(
    TRUE, SM410) THEN
13 INC(TRUE, RobotMaintenance.uSrvOnTimeSec);
14 IF RobotMaintenance.uSrvOnTimeSec = 600 THEN
15 RobotMaintenance.uSrvOnTimeSec := 0;
16 INC(TRUE, RobotMaintenance.uSrvOnTimeMin);
17 IF RobotMaintenance.uSrvOnTimeMin = 60 THEN
18 RobotMaintenance.uSrvOnTimeMin := 0;
19 INC(TRUE, RobotMaintenance.uSrvOnTime);
20 END_IF;
21 END_IF;
22 END_IF;

```

2.6. Bearing Consumption

Bearing consumption is calculated based on the values of currents and motor speeds. The time interval specified in the settings cannot be extended, but it may be shortened. Using the robot at high speeds and with sudden changes in direction will result in a shorter interval. Consumption is counted from the moment Servo On is turned on if the currents are different from zero—consumption can also be counted when the robot is stationary. The function block presented in Listing 4 is responsible for calculating the consumption of bearing.

If the present current is different from zero, the RealTimeMs variable is incremented every 100ms, which counts the operating time of the bearing without any penalties. Then, the percentage of the current value relative to the maximum is calculated. This percentage value is passed as an argument of the quadratic function. For an argument above 20%, the function returns values greater than 100 ms, while for an argument below 20%, it returns the value 100 ms. This means that the consumption increases after exceeding 20% of the current.

Then, the percentage of the current motor speed relative to the maximum is calculated. This percentage value is passed as an argument of the quadratic function. For an argument above 30%, the function returns values greater than 0 ms, while for an argument below 30%, it returns 0 ms. The values of both functions are added together.

The bearing consumption is calculated based on the time generated by the functions. Comparing this time with the actual operating time of the bearing allows us to estimate the replacement interval, based on the current consumption, and to calculate the remaining hours until inspection for such robot operations.

2.7. Gear Prediction

Calculations for detecting gear malfunctions are performed according to the flow diagram presented in Figure 5.

At first, a reference measurement is performed, in which the following are calculated:

- Average of positive current values.
- Average of negative current values.
- Maximum current.
- Minimum current.
- Average positive change in current value.
- Average negative change in current value.
- Maximum positive change in current value.
- Maximum negative change in current value.
- (Average load).
- (Maximum load).

After the reference measurement is completed, the variables that inform us about long-term and short-term malfunctions are calculated. The following are added up for long-term malfunctions: average of positive current values, average of negative current values, average positive change in current value, and average negative change in current value. On the other hand, the following are added up for short-term malfunctions: maximum current, minimum current, maximum positive change in current value, and maximum negative change in current value. Then, the proposed thresholds are calculated. If the reference measurement has been performed, measurements are taken to examine the operation of the gear.

Later measurements are the same as the reference measurement, i.e., calculating certain values for a specified period of time, summing them up appropriately, and comparing them with thresholds. If any of them is exceeded, an appropriate warning is produced.

Listing 4. Consumption bearing function block.

```

1 IntervalSec := INT_TO_DINT(IntervalHours) * 3600;
2
3 IF IntervalSec > 0 AND MaxTolerableCurrent > 0 AND MaxJointSpeed > 0 THEN
4 (*Check if servo is ON*)
5 IF RobotSts.bRobotConnectionStatus AND RobotSts.bRobotDriveStatus AND
   ActualCurrent <> 0 AND LDP(TRUE, SM410) THEN
6 (*Real time counting*)
7 DINC(TRUE, RealTimeMs);
8
9 (*Converting current to percentage of maximum*)
10 ActualCurrentPercent := LREAL_TO_REAL((DINT_TO_LREAL(ABS(ActualCurrent))/
   INT_TO_LREAL(MaxTolerableCurrent))*100.0);
11
12 (*Calculation of the function value*)
13 IF ActualCurrentPercent >= 20.0 THEN
14 FunctionCurrValueMs := (0.0125*((ActualCurrentPercent-20.0)*(
   ActualCurrentPercent-20.0)))+100.0;
15 ELSIF ActualCurrentPercent < 20.0 THEN
16 FunctionCurrValueMs := 100.0;
17 END_IF;
18
19 (*Converting speed to percentage of maximum*)
20 ActualSpeedPercent := LREAL_TO_REAL(DINT_TO_LREAL(ABS(ActualSpeed))/(
   INT_TO_LREAL(MaxJointSpeed)*1000.0)*100.0);
21
22 (*Calculation of the function value*)
23 IF ActualSpeedPercent >= 30.0 THEN
24 FunctionSpeedValueMs := 0.01*((ActualSpeedPercent-30.0)*(
   ActualSpeedPercent-30.0));
25 ELSIF ActualSpeedPercent < 30.0 THEN
26 FunctionSpeedValueMs := 0.0;
27 END_IF;
28
29 (*Adding time result*)
30 TimeMs := TimeMs + FunctionCurrValueMs + FunctionSpeedValueMs;
31 (*Checking for full seconds*)
32 IF TimeMs > 1000.0 THEN
33 DINC(TRUE, TimeSec);
34 TimeMs := TimeMs - 1000.0;
35 END_IF;
36 END_IF;
37
38 (*Calculation of consumption in percentage*)
39 ConsumptionPercent := LREAL_TO_REAL((DINT_TO_LREAL(TimeSec)/DINT_TO_LREAL(
   IntervalSec))*100.0);
40 (*Calculation of hours to service*)
41 IF ConsumptionPercent > 1.0 THEN
42 (*Interval according to previous consumption*)
43 EstIntervalTimeSec := LREAL_TO_DINT(DINT_TO_LREAL(IntervalSec) * (((
   DINT_TO_LREAL(RealTimeMs)/10.0)/DINT_TO_LREAL(TimeSec))));
44
45 (*Remaining hours*)
46 EstRemainHours := LREAL_TO_INT((DINT_TO_LREAL(EstIntervalTimeSec) - (
   DINT_TO_LREAL(RealTimeMs)/10.0)) / 3600.0);
47 ELSE
48 EstRemainHours := IntervalHours;
49 END_IF;
50 END_IF;
51
52 (*Consumption Overhaul Parts*)
53 IF EstRemainHours <= OverhaulPartsTime THEN
54 OverhaulPartsTime := EstRemainHours;
55 OverhaulPartsPercent := ConsumptionPercent;
56 END_IF;

```

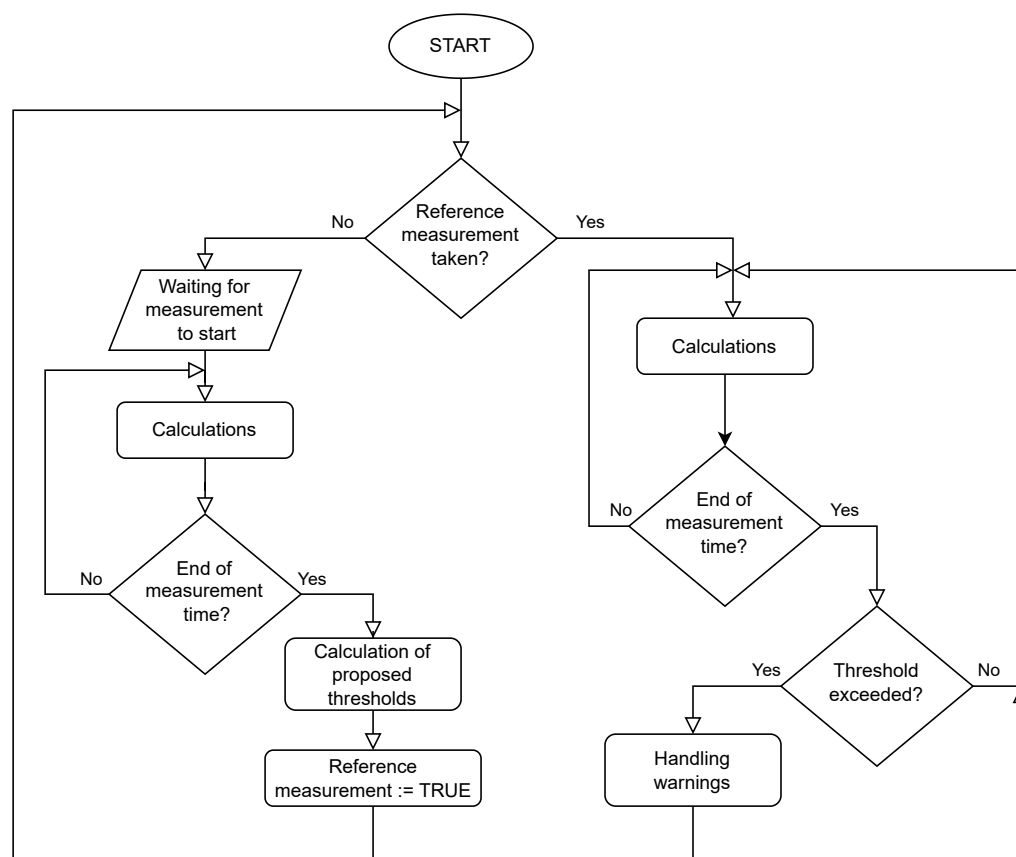


Figure 5. Gear prediction function flow diagram.

3. Results

3.1. Description of Robot Screens

The dedicated screens for operating and monitoring the robot have several common elements. These include a bottom menu and a top panel displaying the most important current statuses of the robot.

The Maintenance menu consists of the following sections:

- Total—overall consumption and tear of the robot.
- MaintParts (maintenance parts)—consumption of the first group of parts.
- OvrhaulParts (overhaul parts)—consumption of the second group of parts.
- AbnormlDetect (abnormalities detection)—detection of abnormalities.
 - Gear J1–J3.
 - Gear J4–J6.
- Log—history of services performed.
 - MaintParts (maintenance parts).
 - OvrhaulParts (overhaul parts).
 - Other.
- OprInf (operating info)—information on operating times.
- Setting—parameter settings.
- Warning—warning threshold settings.
- Reset—deleting alarms and performing services.

3.2. Total

The number of hours worked with Servo On and the use of the Servo On interval in percentages are shown graphically and numerically in Figure 6. Consumption consists

of both part groups. In both cases, the consumption of the part from the group with the lowest time to service is displayed.



Figure 6. Number of hours Servo On exceeded the warning threshold. Maintenance parts exceeded 100%—alarm triggered. Overhaul parts are worn to a degree that does not cause an alarm condition.

3.3. Maintenance Parts

The lubricant and belt consumption are shown in graphical and numerical form in Figure 7. Lubricant consumption is calculated based on the speed of the motors. The interval can become longer or shorter. Consumption is not counted if the motors are not running. Belt consumption is calculated based on the value of currents and the rate of change of the current values. The interval cannot be extended. Using the robot at high speeds and with sudden changes in direction will result in a shorter interval. Consumption is counted from the moment the Servo On is turned on, if the currents are different from zero—consumption can also be counted while the robot is stationary. The estimated time to service is displayed if the consumption exceeds 1%. This is calculated based on the robot's work to date.

3.4. Overhaul Parts

Gearbox and bearing consumption in graphical and numerical form in Figure 8. Gear consumption is calculated on the basis of current values, the rate of change of the current values and the speed of motors. The interval cannot be extended. Using the robot at high speeds and with sudden changes in direction will result in a shorter interval. Consumption is counted from the moment the Servo On is turned on, if the currents are different from zero—consumption can also be counted while the robot is stationary. Bearing consumption is calculated based on the currents and speed of the motors. The interval cannot be extended. Using the robot at high speeds and heavy loads will result in a shorter interval. Consumption is counted from the moment the Servo On is turned on, if the currents are different from zero—consumption can also be counted while the robot is stationary. The estimated time to service is displayed if the consumption exceeds 1%. This is calculated based on the robot's work to date.



Figure 7. Consumption of belts and lubricants.



Figure 8. Consumption of gear and bearing.

3.5. Abnormality Detection

This section has two screens that display the results for detecting gear malfunctions. A reference measurement is required for operation—the longer the better. If one has not been taken, the message “No Reference Measurement” will appear on a red background at the bottom of the screen shown in Figure 9. This can be done by starting the robot in automatic mode and pressing the “Start Reference Measurement” button, which will

change the caption to “During Reference Measurement” during the reference run. When it is completed, the reference result values from the measurement and the suggested detection levels will appear. After a reference run, the abnormality detection function is immediately activated. The robot’s operation is analyzed, and the results are analyzed at certain time intervals. If the results exceed the abnormality detection level, a warning is reported, which is made cleared with the physical Reset button. Abnormalities are divided into those that have been detected for most of the examined time, “Long-term”, and temporary, “Short-term”, disturbances in the gear system. Detection levels can be modified by the user. The results are stored and displayed from the last three detection periods. If any of them exceeds the detection level, its background turns yellow. Results from the current sample are displayed, for example, one minute after the start of the period. The values of the long-term results can increase as well as decrease during the ongoing analysis. Short-term results, on the other hand, do not decrease during the test sample. The result is calculated based on the average and maximum value of currents and the rate and magnitude of current changes. The results vary depending on the work performed by the robot. So, the threshold should be set for the current working conditions of the robot.



Figure 9. J4–J6 joints. The reference run took place. This follows several measurements analyzing the robot’s operation.

3.6. LOG

The screen presented in Figure 10 shows how many services have been performed on a particular part, at what consumption, and when the last replacement/overhaul was performed.



Figure 10. The first of the log screens—belts and greases.

3.7. Operating Info

Figure 11 shows the times since the last full inspection of the robot controller was turned on, the time since the servo was engaged, and the time during which the robot moved. In addition, it shows the number of servo engagements. The right side of the screen displays the total motor times of each joint.

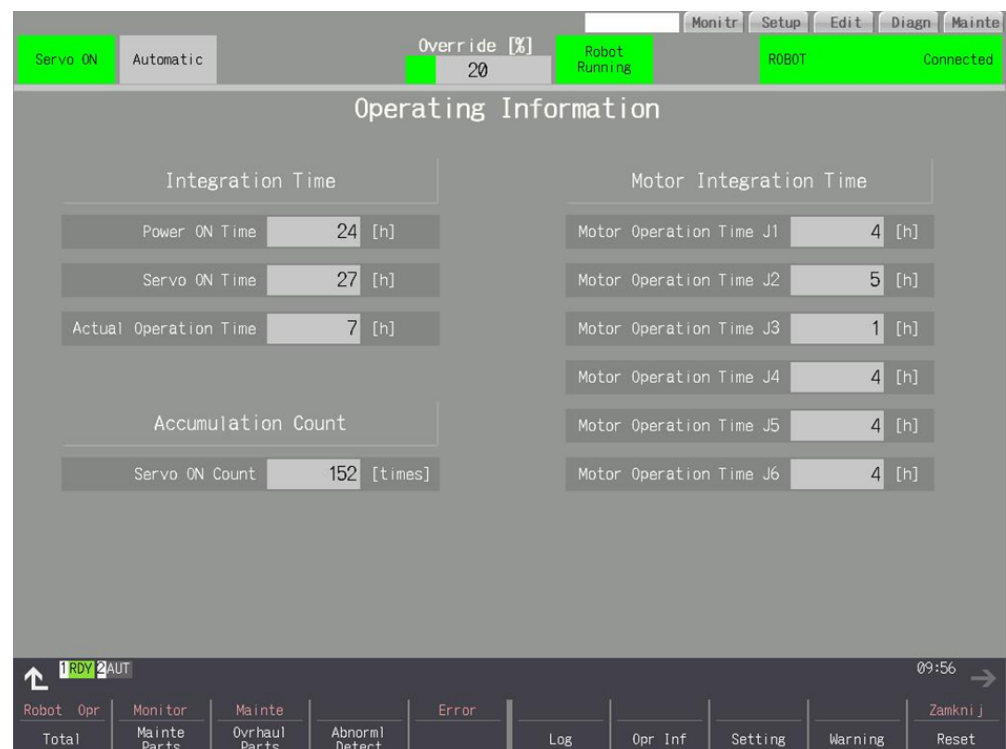


Figure 11. Total times since last full inspection.

3.8. Settings

Configurable consumption levels of given parts at which a warning is to be reported (10%–99%)—presented in Figure 12. The warning is a yellow message with the exact contents and the numerical value of the consumption and the remaining hours to be serviced for a given part highlighted in yellow. The warning appears when the CNC panel is turned on again—presented in Figure 13.



Figure 12. Warning levels.

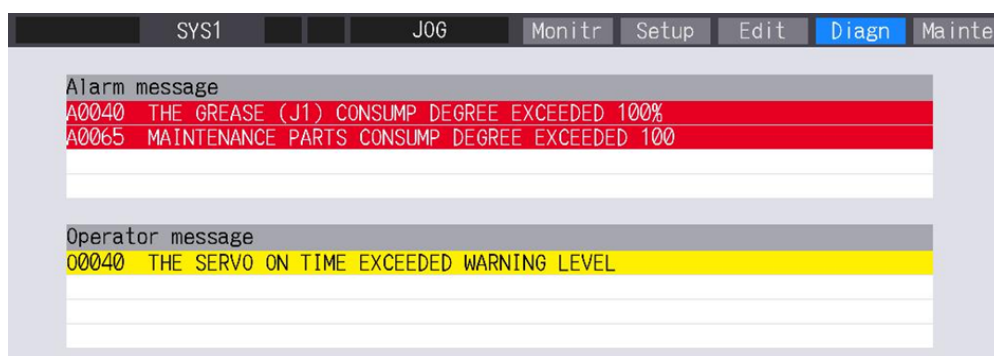


Figure 13. Warning samples.

3.9. Reset

A screen for clearing alarms and logging the completed inspections is presented in Figure 14. Alarms are represented by a red message with the exact contents and with red highlighting the numerical value of the consumption and the remaining hours to overhaul a given part after exceeding 100% consumption. Resetting the alarm only removes the message. The alarm appears when the CNC panel is turned on again. The reset records the date and current consumption of the part, which is displayed on the log screen.

- Grease—reset of grease consumption.
- Timing belt—reset of belt consumption.
- Reduction gear—reset of gear consumption.
- Bearing—reset of bearing consumption.
- Encoder—reset of motor timing.
- Overhaul, mechanical—reset of all consumption and times.

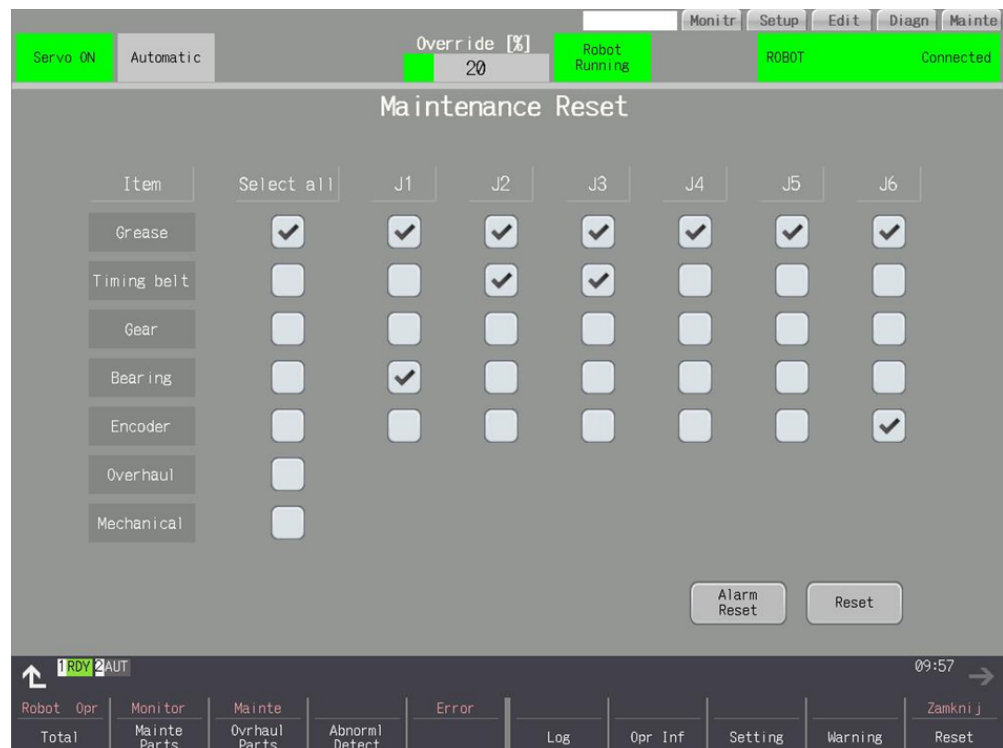


Figure 14. Clearing alarms and recording inspections performed.

4. Conclusions

An original implementation of an industrial robot control system was presented. The solution was based on the DRC mechanism and multi-channel synchronization in the CNC. Dedicated programs were developed in the PLC for diagnostics based on numerical data from the industrial robot. In combination with the HMI screens, they allow for quick and efficient diagnostics of the robot during the production of parts on a lathe operated by the aforementioned industrial robot. Detecting potential failures before they occur allows for planning maintenance without disrupting the production process. This prevents expensive failures and unplanned downtime, which translate into savings for the end customer. Regular maintenance based on the actual condition of the robot extends its life and work efficiency. The collected data are analyzed in the PLC. The results are presented on the HMI screen of the CNC and also sent to the computer, where they can be made available to superior systems thanks to IIoT technology. The solution was tested on a real industrial station, where the robot picked up parts from the magazine, fed them to the lathe, where the machining was then carried out in accordance with the specified technology.

The presented solutions were prepared for Mitsubishi Electric robots controlled by DRC via a CNC. It is possible to extend the scope of application of the solution for FANUC robots also controlled by DRC via a Mitsubishi CNC. In the case of other robot manufacturers, it is possible to use the same mechanisms while ensuring data delivery using alternative data protocols supported by the considered industrial robots. The extension of functionality can

also be implemented within the mechanisms of data collection on the PC side and their further analysis in order to detect additional anomalies in the work of the station.

Author Contributions: Conceptualization, P.C. and A.W.; methodology, P.C. and A.W.; software, P.C. and A.W.; validation, P.C. and A.W.; formal analysis, P.C. and A.W.; investigation, P.C. and A.W.; resources, P.C. and A.W.; data curation, P.C. and A.W.; writing—original draft preparation, P.C. and A.W.; writing—review and editing, P.C. and A.W.; visualization, P.C. and A.W.; supervision, P.C. and A.W.; project administration, A.W.; funding acquisition, P.C. and A.W. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Institute of Control and Computation Engineering, Faculty of Electronics and Information Technology, Warsaw University of Technology.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Upon request from the authors.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Izagirre, U.; Andonegui, I.; Landa-Torres, I.; Zurutuza, U. A practical and synchronized data acquisition network architecture for industrial robot predictivemaintenance in manufacturingassembly lines. *Robot. Comput.-Integr. Manuf.* **2022**, *74*, 102287.
2. Pinto, R.; Cerquitelli, T. Robot fault detection and remaining life estimation for predictive maintenance. In Proceedings of the 2nd International Conference on Emerging Data and Industry 4.0 (EDI40), Leuven, Belgium, 29 April–2 May 2019.
3. Nentwich, C.; Reinhart, G. Towards Data Acquisition for Predictive Maintenance of Industrial Robots. In Proceedings of the 54th CIRP Conference on Manufacturing Systems, Patras, Greece, 22–24 September 2021.
4. Doyel, J.; Gallege, T.; Ebru, T.B.; Dudas, C.; Skoogh, A. A Predictive Maintenance Application for A Robot Cell using LSTM Model. *IFAC PapersOnLine* **2022**, *55*, 115–120.
5. Koca, O.; Kaymakci, T.O.; Mercimek, M. Advanced Predictive Maintenance with Machine Learning Failure Estimation in Industrial Packaging Robots. In Proceedings of the 15th International Conference on Development and Application Systems, Suceava, Romania, 21–23 May 2020.
6. Borgi, T.; Hidri, A.; Neef, B.; Saber, M.N. Data Analytics for Predictive Maintenance of Industrial Robots. In Proceedings of the International Conference on Advanced Systems and Electric Technologies (IC_ASET), Hammamet, Tunisia, 14–17 January 2017.
7. Kim, H.G.; Yoon, H.S.; Yoo, J.H.; Yoon, H.I.; Han, S.S. Development of Predictive Maintenance Technology for Wafer Transfer Robot using Clustering Algorithm. In Proceedings of the International Conference on Electronics, information, and Communication(ICEIC), Auckland, New Zealand, 22–25 January 2019.
8. Mourtzis, D.; Tsoubou, S.; Angelopoulos, J. Robotic Cell Reliability Optimization Based on Digital Twin and Predictive Maintenance. *Electronics* **2023**, *12*, 1999. [[CrossRef](#)]
9. Morettini, S. Machine Learning in Predictive Maintenance of Industrial Robots. Master’s Thesis, KTH, Stockholm, Sweden, 2021; Degree Project in Computer Science and Engineering.
10. Bonc,i A.; Longhi, S.; Nabissi, G.; Verdini, F. Predictive Maintenance System using motor current signal analysis for Industrial Robot. In Proceedings of the 24th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA), Zaragoza, Spain, 10–13 September 2019.
11. Staneliff, S.; Dolan, J.; Trebi-Ollennu, A. *Towards a Predictive Model of Mobile Robot Reliability*; CMU-RI-TR-05-38; Carnegie Mellon University: Pittsburgh, PA, USA, 2005.
12. Mei, B.; Xie, F.; Liu, X.-J.; Li, H. Calibration of a 6-DOF industrial robot considering the actual mechanical structures and CNC system. In Proceedings of the 2nd International Conference on Robotics and Automation Engineering (ICRAE), Shanghai, China, 29–31 December 2017; pp. 6–10.
13. Chang, R.-L.; Han, J.; Cui, G.-W.; Gao, M.-X.; Wei, Y.-L. Workspace Analysis of CNC Load-Unload Material Robot. In Proceedings of the 2nd World Conference on mechanical Engineering and Intelligent Manufacturing (WCMEIM), Shanghai, China, 22–24 November 2019; pp. 456–460.
14. Ramadiansyah, M.; Wahab, W.; Nasril. Modelling, simulation and control of a high precision loading-unloading robot for CNC milling machine. In Proceedings of the 15th International Conference on Quality in Research (QiR): International Symposium on Electrical and Computer Engineering, Nusa Dua, Bali, Indonesia, 24–27 July 2017; pp. 204–208.

15. Lee, S.-D.; Ahn, K.-H.; Song, J.-B. Torque control based sensorless hand guiding for direct robot teaching. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Daejeon, Republic of Korea, 9–14 October 2016; pp. 745–750.
16. Hügler, J.; Lambrecht, J.; Krüger, J. An integrated approach for industrial robot control and programming combining haptic and non-haptic gestures. In Proceedings of the 26th IEEE International Symposium on Robot and Human Interactive Communication (RO-MAN), Lisbon, Portugal, 28 August–1 September 2017; pp. 851–857.
17. Rath, G.; Probst, G.; Kollment, W. Robot control unit for educational purposes. In Proceedings of the International Conference on Applied Electronics (AE), Pilsen, Czech Republic, 8–9 September 2015; pp. 205–208.
18. Atmoko, R.A.; Yang, D. Online Monitoring & Controlling Industrial Arm Robot Using MQTT Protocol. In Proceedings of the IEEE International Conference on Robotics, Biomimetics, and Intelligent Computational Systems (Robionetics), Bandung, Indonesia, 8–10 August 2018; pp. 12–16.
19. Ghobakhloo, M. Industry 4.0, digitization, and opportunities for sustainability. *J. Clean. Prod.* **2020**, *252*, 119869. [[CrossRef](#)]
20. Mukh, i M.; Portugal, ; D.; Pereira, S.; Couceiro, M.S. A novel solution for securing robot communications based on the MQTT protocol and ROS. In Proceedings of the IEEE/SICE International Symposium on System Integration (SII), Paris, France, 14–16 January 2019; pp. 608–613.
21. Mizuya, T.; Okuda, M.; Nagao, T. A case study of data acquisition from field devices using OPC UA and MQTT. In Proceedings of the 56th Annual Conference of the Society of Instrument and Control Engineers of Japan (SICE), Kanazawa, Japan, 19–22 September 2017; pp. 611–614.
22. Swami, M.; Verma, D.; Vishwakarma, V.P. Blockchain and Industrial Internet of Things: Applications for Industry 4.0. In *Proceedings of the International Conference on Artificial Intelligence and Applications; Advances in Intelligent Systems and Computing*; Springer: Singapore, 2021; Volume 1164.
23. Shi, H.; Niu, L.; Sun, J. Construction of Industrial Internet of Things Based on MQTT and OPC UA Protocols. In Proceedings of the IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA), Dalian, China, 27–29 June 2020; pp. 1263–1267.
24. Jaloudi, S. Communication Protocols of an Industrial Internet of Things Environment: A Comparative Study. *Future Internet* **2019**, *11*, 66. [[CrossRef](#)]
25. Rodríguez, M.; Tobon, P.D.; Munera, D. A framework for anomaly classification in Industrial Internet of Things systems. *Internet Things* **2025**, *29*, 101446. [[CrossRef](#)]
26. Calderon, D.; Folgado, F.J.; Gonzalez, I.; Calderon, A.J. Implementation and Experimental Application of Industrial IoT Architecture Using Automation and IoT Hardware/Software. *Sensors* **2024**, *24*, 8074. [[CrossRef](#)] [[PubMed](#)]
27. Anitha, T.; Manimurugan, S.; Sridhar, S.; Mathupriya, S.; Latha, G.C.P. A Review on Communication Protocols of Industrial Internet of Things. In Proceedings of the 2nd International Conference on Computing and Information Technology (ICCIT), Tabuk, Saudi Arabia, 30 November–2 December 2022; pp. 418–423.
28. Khan, W.Z.; Rehman, M.H.; Zangoti, H.M.; Afzal, M.K.; Armi, N.; Salah, K. Industrial internet of things: Recent advances, enabling technologies and open challenges. *Comput. Electr. Eng.* **2020**, *81*, 106522. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.